

Two Pointers — Complete Revision

All problems solved together, patterns, pointer rules, mistakes, dry-run intuition, and final Java code

Goal: Recognize the Two Pointer pattern in a new interview problem instead of memorizing individual solutions.

#	Problem	Pattern
26	Remove Duplicates from Sorted Array	Same direction / read-write
27	Remove Element	Same direction / read-write
283	Move Zeroes	Same direction / read-write
125	Valid Palindrome	Opposite direction
167	Two Sum II	Sorted pair / opposite direction
11	Container With Most Water	Opposite / limiting pointer
977	Squares of a Sorted Array	Two ends + result array
344	Reverse String	Opposite / swap
345	Reverse Vowels of a String	Opposite / conditional swap

1. What is Two Pointers?

- **Two Pointer** means using two indexes to process an array or string efficiently.
- Pointers can move from the same side (read/write) or from opposite sides.
- The main benefit is avoiding repeated work and often reducing $O(n^2)$ brute force to $O(n)$.
- **Fast/read**: explores input. **Slow/write**: stores valid elements.
- **Opposite pointers**: compare, swap, or use information from both ends.

2. Same Direction — Read/Write

```
int j = 0;

for (int i = 0; i < nums.length; i++) {
    if (condition) {
        nums[j] = nums[i];
        j++;
    }
}
```

- i = fast/read pointer; scans the input.
- j = slow/write pointer; points to the next place for a valid element.
- Memory trick: **Fast reads, slow writes**.
- Used in Remove Duplicates, Remove Element, and Move Zeroes.

3. Opposite Direction

```
int left = 0;
int right = nums.length - 1;

while (left < right) {
    // compare / swap / calculate
    // move one or both pointers
}
```

- Pointers start at the two ends and move toward the center.
- Used for palindrome, reverse, sorted pair, and container problems.
- The key interview skill is explaining **why** a pointer moves.

4. When should I think Two Pointers?

- **Sorted array + pair/sum/closest**: strong Two Pointer signal.
- **Palindrome or reverse**: strong opposite-direction signal.
- **In-place remove/filter/rearrange**: strong same-direction signal.
- **Contiguous subarray/substring**: usually think Sliding Window next.
- Sorted array alone does not guarantee Two Pointers; searching for one value may be Binary Search.

5. How do I decide which pointer moves?

- **Two Sum II**: sum too small $\rightarrow$ left++; sum too large $\rightarrow$ right--.
- **Container**: move the shorter height because it limits the area.
- **Palindrome/reverse**: after processing, move both.
- **Conditional swap**: advance only the pointer sitting on an irrelevant character.

6. < vs <= — Off-by-one rule

- **left < right:** use when you need two different positions. Examples: Two Sum II, Container, Palindrome, Reverse.
- **left <= right:** use when the same remaining position still needs processing. Example: Squares of a Sorted Array.
- Ask: **When left == right, does this element still need work?**

26. Remove Duplicates from Sorted Array

Pattern: Same direction / read-write

Question: Remove duplicates in-place from a sorted array and return the number k of unique elements.

Core idea: Keep the first unique element. Scan with i; if `nums[i]` differs from the last stored unique value `nums[j-1]`, write it to `nums[j]` and increment j.

Final Java code

```
class Solution {
    public int removeDuplicates(int[] nums) {

        if (nums.length == 0)
            return 0;

        int j = 1;

        for (int i = 1; i < nums.length; i++) {

            if (nums[i] != nums[j - 1]) {
                nums[j] = nums[i];
                j++;
            }
        }

        return j;
    }
}
```

Dry run / intuition

- [1,1,2]: i=1 sees another 1 → skip.
- i=2 sees 2 != last unique 1 → write 2 at index 1; j becomes 2.
- Return 2; first two positions are [1,2].

Common mistakes

- Using `i < nums.length - 1` skips the last element.
- Starting `j=1` without handling an empty array can fail.
- Elements after k are irrelevant.

Complexity: $O(n)$ time, $O(1)$ extra space.

27. Remove Element

Pattern: Same direction / read-write

Question: Remove all occurrences of val in-place and return the number k of elements not equal to val.

Core idea: Scan with i. If nums[i] != val, copy it to nums[j] and increment j.

Final Java code

```
class Solution {
    public int removeElement(int[] nums, int val) {

        int j = 0;

        for (int i = 0; i < nums.length; i++) {

            if (nums[i] != val) {
                nums[j] = nums[i];
                j++;
            }
        }

        return j;
    }
}
```

Dry run / intuition

- For [3,2,2,3], val=3: skip the first 3.
- Copy the two 2s to positions 0 and 1.
- Skip the final 3 and return 2.

Common mistakes

- Do not use nums.length - 1 as the number of elements.
- Starting i=1 skips the first element.
- k is a count, not the last index.

Complexity: O(n) time, O(1) extra space.

283. Move Zeroes

Pattern: Same direction / read-write + fill

Question: Move all zeroes to the end while preserving the relative order of non-zero elements.

Core idea: Copy all non-zero values forward using j, then fill the remaining positions with zero.

Final Java code

```
class Solution {
    public void moveZeroes(int[] nums) {

        int j = 0;

        for (int i = 0; i < nums.length; i++) {
            if (nums[i] != 0) {
                nums[j] = nums[i];
                j++;
            }
        }

        while (j < nums.length) {
            nums[j] = 0;
            j++;
        }
    }
}
```

Dry run / intuition

- For [0,1,0,3,12], non-zero values become [1,3,12,...].
- After scanning, j=3, so indexes 3 and 4 are filled with zero.
- Result: [1,3,12,0,0].

Common mistakes

- For arrays use nums.length, not nums.length().
- Opposite pointers are unnecessary here.
- j always means the next write position.

Complexity: O(n) time, O(1) extra space.

125. Valid Palindrome

Pattern: Opposite direction

Question: Check whether a string is a palindrome after ignoring non-alphanumeric characters and case.

Core idea: Skip irrelevant characters from both ends, compare lowercase valid characters, and return false on mismatch.

Final Java code

```
class Solution {
    public boolean isPalindrome(String s) {

        int left = 0;
        int right = s.length() - 1;

        while (left < right) {

            while (left < right &&
                !Character.isLetterOrDigit(s.charAt(left))) {
                left++;
            }

            while (left < right &&
                !Character.isLetterOrDigit(s.charAt(right))) {
                right--;
            }

            if (Character.toLowerCase(s.charAt(left)) !=
                Character.toLowerCase(s.charAt(right))) {
                return false;
            }

            left++;
            right--;
        }

        return true;
    }
}
```

Dry run / intuition

- Skip spaces/punctuation from each side.
- Compare only valid characters after lowercasing.
- If all comparisons match, return true.

Common mistakes

- Forgetting to skip punctuation/spaces.
- Comparing case-sensitively.
- Moving both pointers before finding valid characters.

Complexity: $O(n)$ time, $O(1)$ extra space.

167. Two Sum II — Input Array Is Sorted

Pattern: Sorted pair / opposite direction

Question: Find two numbers in a sorted array that add to target and return their 1-based indices.

Core idea: Start at both ends. Sum too small $\rightarrow$ left++; sum too large $\rightarrow$ right--.

Final Java code

```
class Solution {
    public int[] twoSum(int[] numbers, int target) {

        int left = 0;
        int right = numbers.length - 1;

        while (left < right) {

            int sum = numbers[left] + numbers[right];

            if (sum == target) {
                return new int[]{left + 1, right + 1};
            }

            if (sum < target) {
                left++;
            } else {
                right--;
            }
        }

        return new int[]{};
    }
}
```

Dry run / intuition

- [2,7,11,15], target=9: 2+15=17 $\rightarrow$ right--.
- 2+11=13 $\rightarrow$ right--.
- 2+7=9 $\rightarrow$ found.

Common mistakes

- This works because the array is sorted.
- LeetCode asks for 1-based indices.
- Never use the same index twice.

Complexity: $O(n)$ time, $O(1)$ extra space.

11. Container With Most Water

Pattern: Opposite direction / limiting pointer

Question: Choose two lines that form the container holding the most water.

Core idea: Area = width × minimum height. Move the shorter-height pointer because it is the limiting side.

Final Java code

```
class Solution {
    public int maxArea(int[] height) {

        int left = 0;
        int right = height.length - 1;
        int maxArea = 0;

        while (left < right) {

            int width = right - left;
            int minHeight = Math.min(height[left], height[right]);
            int currentArea = width * minHeight;

            maxArea = Math.max(maxArea, currentArea);

            if (height[left] < height[right]) {
                left++;
            } else {
                right--;
            }
        }

        return maxArea;
    }
}
```

Dry run / intuition

- Start with the widest container.
- Area is limited by the shorter height.
- Moving the taller side only reduces width without fixing the limiting side, so move the shorter side.

Common mistakes

- Moving the wrong pointer.
- Using the taller height instead of min(...).
- Using nested loops instead of O(n) pointers.

Complexity: O(n) time, O(1) extra space.

977. Squares of a Sorted Array

Pattern: Two ends + result array

Question: Return squares in non-decreasing order from an already sorted array.

Core idea: The largest square can come from either end. Put the larger square at the back of result and move that side.

Final Java code

```
class Solution {
    public int[] sortedSquares(int[] nums) {

        int left = 0;
        int right = nums.length - 1;

        int[] result = new int[nums.length];
        int k = nums.length - 1;

        while (left <= right) {

            int leftSquare = nums[left] * nums[left];
            int rightSquare = nums[right] * nums[right];

            if (leftSquare > rightSquare) {
                result[k] = leftSquare;
                left++;
            } else {
                result[k] = rightSquare;
                right--;
            }

            k--;
        }

        return result;
    }
}
```

Dry run / intuition

- For [-7,-3,2,3,11], compare 49 and 121 → put 121 at result[k].
- Keep filling from the back.
- When left==right, the final element still needs processing, so use <=.

Common mistakes

- Using left < right leaves one element unprocessed.
- Forgetting k-- after both branches.
- Using nums.length() instead of nums.length.

Complexity: O(n) time, O(n) extra space for result.

344. Reverse String

Pattern: Opposite direction / swap

Question: Reverse a character array in-place.

Core idea: Swap the two ends, then move both pointers inward.

Final Java code

```
class Solution {
    public void reverseString(char[] s) {

        int left = 0;
        int right = s.length - 1;

        while (left < right) {

            char temp = s[left];
            s[left] = s[right];
            s[right] = temp;

            left++;
            right--;
        }
    }
}
```

Dry run / intuition

- Swap first/last, then move both inward.
- Stop when they meet/cross; the middle character needs no swap.

Common mistakes

- Using `<=` is unnecessary.
- Forgetting to increment/decrement after the swap.

Complexity: $O(n)$ time, $O(1)$ extra space.

345. Reverse Vowels of a String

Pattern: Opposite direction / conditional swap

Question: Reverse only vowels while keeping all consonants in place.

Core idea: Use a mutable char array. Move each pointer until it reaches a vowel; swap vowels and then move both.

Final Java code

```
class Solution {
    public String reverseVowels(String s) {

        char[] ch = s.toCharArray();

        int left = 0;
        int right = ch.length - 1;

        while (left < right) {

            while (left < right && !isVowel(ch[left])) {
                left++;
            }

            while (left < right && !isVowel(ch[right])) {
                right--;
            }

            char temp = ch[left];
            ch[left] = ch[right];
            ch[right] = temp;

            left++;
            right--;
        }

        return new String(ch);
    }

    private boolean isVowel(char c) {
        return c == 'a' || c == 'e' || c == 'i' ||
            c == 'o' || c == 'u' ||
            c == 'A' || c == 'E' || c == 'I' ||
            c == 'O' || c == 'U';
    }
}
```

Dry run / intuition

- For IceCreAm, find I from the left and A from the right, then swap.
- Skip consonants by moving only the pointer that is on a consonant.
- Return after the loop; returning inside the loop stops after one swap.

Common mistakes

- Putting return inside while.
- Forgetting uppercase vowels.
- String is immutable; char[] or StringBuilder is needed for swapping.

Complexity: O(n) time, O(n) extra space for char[]. StringBuilder is also a valid mutable alternative.

Final Two Pointer Cheat Sheet

Use this before starting a new DSA problem.

Situation	Pattern	Rule
In-place remove/filter	Same direction	fast reads; slow writes
Sorted + pair/sum	Opposite direction	small sum → left++; large sum → right--
Palindrome	Opposite direction	skip invalid; compare; move both
Reverse	Opposite direction	swap; move both
Container	Opposite direction	move the shorter height
Sorted squares	Two ends + result	larger square goes to back; k--
Conditional swap	Opposite + skip	advance until relevant element

Recognition checklist

- Array/string? Consider pointers.
- Two ends, palindrome, reverse, pair? Consider opposite pointers.
- In-place filtering/removal? Consider slow/write + fast/read.
- Sorted + pair/sum? Consider opposite pointers.
- Contiguous subarray/substring? Consider Sliding Window.
- Always explain why a pointer moves.
- Check whether left == right still needs processing.

Completed together

LeetCode 26, 27, 283, 125, 167, 11, 977, 344, and 345. These cover the major Two Pointer patterns we worked through. The next major DSA pattern is Sliding Window.

Golden memory line: Fast reads, slow writes. Two ends move inward. Always know WHY a pointer moves.